Do problema ao software

Guia de estudo: engenharia de software orientada a objetos, explicada com o projeto Maple Seed · 30 de outubro de 2026

Sumário

1. A ideia geral: por que tantas etapas? 1
2. Elicitação de requisitos 2
3. Atores, cenários e casos de uso 2
4. Por que UML? 3
5. Diagrama de casos de uso 4
6. Diagrama de classes (modelo de objetos) 5
7. Diagrama de sequência 8
8. Máquinas de estado 9
9. Arquitetura 11
10. O fio que liga tudo: rastreabilidade 14
Resumo para revisão 14

1. A ideia geral: por que tantas etapas?

Mudar um desenho é barato; mudar código pronto é caro. As etapas existem para descobrir erros quando eles ainda custam pouco.

Pense na construção de uma casa. Ninguém começa assentando tijolo. Primeiro você conversa com o dono ("quantos quartos? tem criança? trabalha de casa?"), depois o arquiteto faz a planta, o engenheiro calcula a estrutura, a parte elétrica e a hidráulica, e só então vem a obra, a vistoria e a manutenção.

Software é igual. Um erro descoberto na conversa inicial custa uma frase; o mesmo erro descoberto depois do sistema construído pode custar semanas. Cada etapa responde a uma pergunta diferente:

Etapa	Pergunta que responde	No Maple Seed
1. Elicitação de requisitos	O que o cliente precisa?	Hipóteses de dor, telas do protótipo, casos de uso
2. Análise	Que conceitos e comportamentos o sistema precisa ter, vistos do lado do usuário?	Diagrama de classes, sequência e estados
3. Design de alto nível (arquitetura)	Como dividir o sistema em partes, e onde cada parte roda?	Três camadas, cinco subsistemas
4. Design detalhado de objetos	Como fica cada classe no código?	Próximo passo, depois de escolher a stack
5. Implementação	Escrever o código	O MVP, um caso de uso por vez
6. Testes	O código faz o que os requisitos pediam?	Testes derivados dos casos de uso
7. Implantação e manutenção	Colocar no ar e manter funcionando	Demo publicada, atualização dos dados

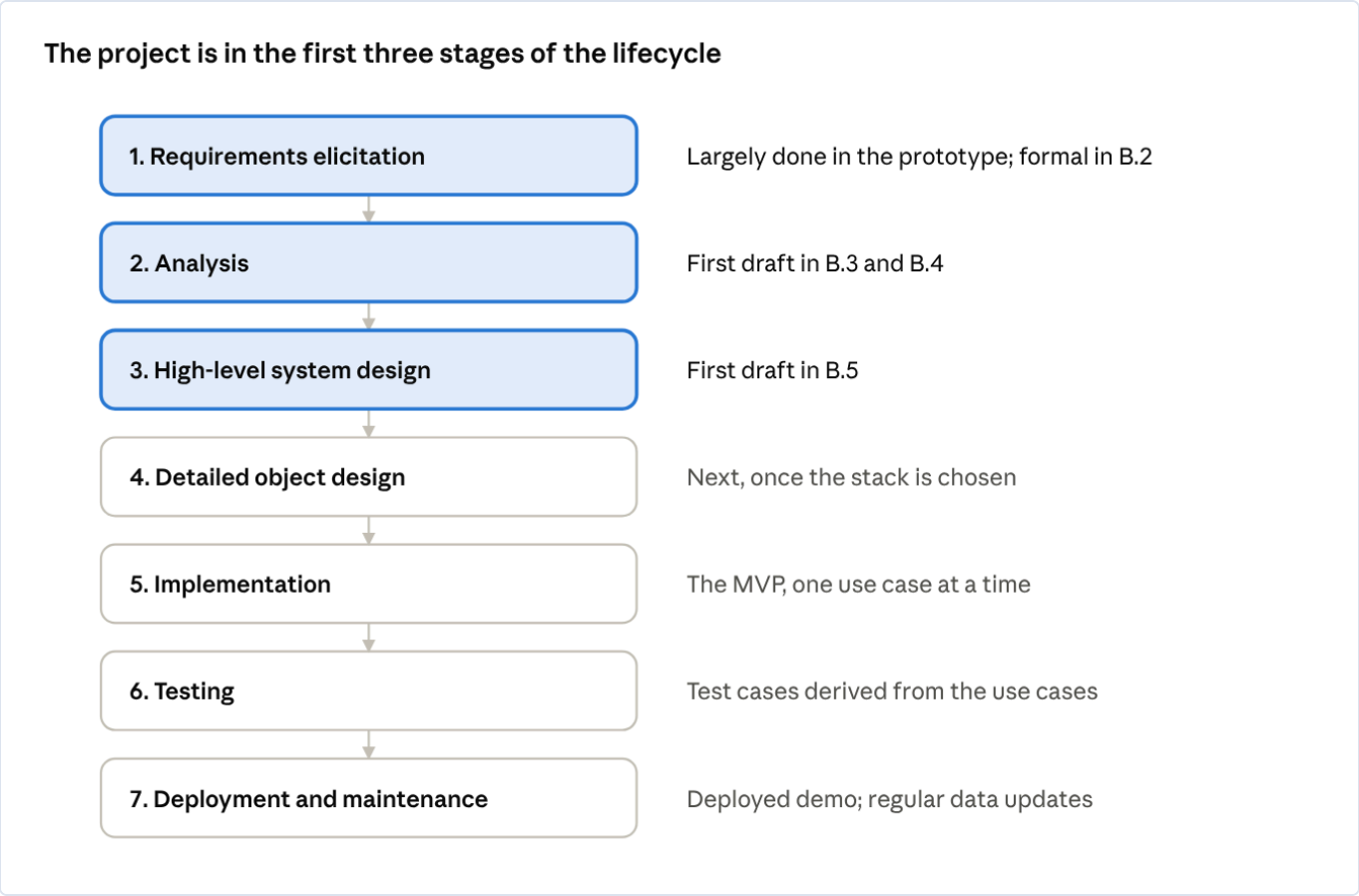


Figura 1. As sete etapas e onde o Maple Seed está hoje.

Um detalhe importante: na prática isso não é uma fila que anda só para a frente. Você vai e volta. O protótipo de telas, por exemplo, já foi uma forma de fazer a etapa 1, e ao escrever os casos de uso descobrimos um usuário que estava faltando. Isso é normal e saudável.

2. Elicitação de requisitos

"Elicitar" vem do latim *elicere*, "tirar para fora". O cliente quase nunca sabe dizer de cara o que precisa: ele sabe a dor, não a solução.

Por isso a palavra é elicitar, e não "coletar". O trabalho é puxar as necessidades para fora com conversas, cenários, protótipos e perguntas. O resultado são **requisitos**, de dois tipos:

- **Funcionais:** o que o sistema faz. Exemplo: "gerar um PDF da lista salva".
- **Não funcionais:** como ele precisa ser. Exemplos: "bilíngue", "acessível", "resposta em menos de um segundo", "respeitar a lei de privacidade do Québec".

Os não funcionais são os mais esquecidos e muitas vezes os que mais pesam. Ser bilíngue desde o início, por exemplo, muda como você guarda cada texto no banco de dados.

3. Atores, cenários e casos de uso

Ator

Ator é quem está **fora** do sistema e interage com ele: uma pessoa ou outro sistema. O que define um ator é o **papel**, não a pessoa. No Maple Seed, você mesma pode ser Visitante quando testa o site e Curadora quando atualiza os programas: são dois atores, mesmo sendo uma pessoa só.

Cenário

Cenário é uma história concreta, com nome e detalhes: "Maria, dona de uma empresa de alimentos com 12 funcionários em Ontario, filtra por Ontario e Agro, lê dois programas e clica no site oficial". É fácil de entender e ótimo para conversar com quem não é técnico.

Caso de uso

Caso de uso é a generalização de vários cenários parecidos: "Buscar programas". Ele não tem a Maria; tem "o visitante". A relação entre os dois é como a entre uma receita (caso de uso) e o jantar de ontem (cenário). Cada caso de uso tem:

- **Ator que inicia:** quem começa a interação.
- **Condição de entrada:** o que precisa ser verdade antes.
- **Fluxo de eventos:** os passos.
- **Condição de saída:** o que é verdade depois.
- **Fluxos alternativos:** e se der errado, ou se o usuário voltar?

Fronteira do sistema

É a linha entre o que você constrói e o que fica de fora. No Maple Seed, decidimos que a candidatura ao programa fica **fora**: o site só leva até o site oficial. Essa linha, desenhada cedo, protege o projeto de crescer demais e de riscos legais.

---

4. Por que UML?

UML (Unified Modeling Language) é uma linguagem padrão de desenhos para software: qualquer engenheiro no mundo lê um losango preenchido do mesmo jeito.

Os diagramas servem para três coisas:

1. **Pensar:** desenhar obriga você a decidir coisas que no texto ficam vagas.
2. **Comunicar:** uma pessoa entende a estrutura do sistema em minutos, sem ler código.
3. **Encontrar erros cedo:** furos aparecem no papel, quando ainda é barato consertar.

Eles se dividem em dois grupos: os **estruturais** mostram o que existe (classes, componentes) e os **comportamentais** mostram o que acontece ao longo do tempo (sequências, estados). Foi feito um de cada tipo que o professor citou, explicados a seguir.

5. Diagrama de casos de uso

Mostra quem usa o sistema e para fazer o quê. É o mapa de funcionalidades.

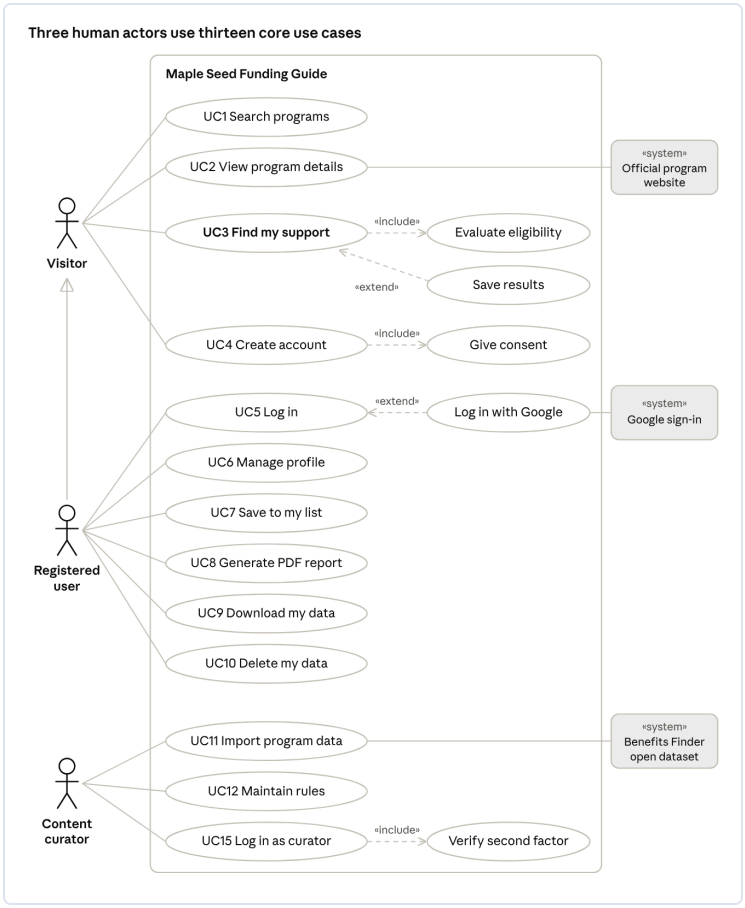


Figura 2. Diagrama de casos de uso do Maple Seed (escopo do MVP).

Por que estas escolhas

- **Três atores humanos (Visitante, Usuário registrado, Curador).** Visitante e Registrado vieram das telas. O Curador apareceu ao perguntar "quem coloca os programas no sistema?". As telas não respondiam a isso, e é esse tipo de descoberta que justifica o método.
- **Herança (triângulo vazio) de Registrado para Visitante.** Tudo que um visitante faz, o registrado também faz. Sem a herança, seria preciso ligar o Registrado a todos os casos do Visitante, e o desenho viraria um emaranhado.
- **«include» em "Avaliar elegibilidade".** O Find My Support **sempre** avalia elegibilidade, e a busca com "usar meu perfil" também. Separar o "avaliar" deixa claro que é um pedaço reutilizado.
- **«extend» em "Salvar resultados" e "Login com Google".** São **opcionais**: só acontecem às vezes.
- **UC15 "Log in as curator", com «include» em "Verify second factor".** Este caso de uso apareceu depois, ao perguntar "como o curador entra no sistema?". A verificação em duas etapas é um include porque acontece **sempre** que um curador entra.

Para a prova: include = sempre acontece (a seta sai do caso base); extend = acontece só às vezes (a seta sai da extensão e aponta para o caso base).

6. Diagrama de classes (modelo de objetos)

Mostra os conceitos do problema (classes), o que cada um sabe (atributos), o que cada um faz (operações) e como eles se relacionam.

Como cheguei a essas classes

As classes não foram inventadas do nada: elas foram **extraídas do texto** que já tínhamos (cenários, casos de uso, telas). A técnica, ensinada no livro do Bruegge, se chama **análise textual** (ou método de Abbott). A ideia é simples: as palavras do texto já dizem quase tudo.

No texto aparece...	Normalmente vira...	Exemplo
Substantivo importante	Classe candidata	"programa", "critério", "lista"
Substantivo que é só um valor	Atributo	"valor máximo", "província", "e-mail"
Verbo (o que se faz)	Operação	"avaliar", "salvar", "excluir"
"tem", "contém", "pertence a"	Associação (e talvez composição ou agregação)	"o programa **tem** critérios"
"é um tipo de"	Herança	"o critério de província **é um tipo de** critério"

Passo 1: sublinhar os substantivos

Peguei frases dos casos de uso e dos cenários. Por exemplo, o fluxo do Find My Support: "O sistema avalia o perfil da empresa contra os critérios publicados de cada programa ativo e mostra os resultados agrupados, cada um com o motivo por critério." E também: "O usuário salva programas na sua lista e gera um relatório PDF", "O usuário dá seu consentimento ao criar a conta", "Cada programa é oferecido por uma agência e tem um valor máximo e um site oficial".

Passo 2: filtrar os candidatos

Nem todo substantivo vira classe. Para cada um, fiz quatro perguntas: (1) o sistema precisa **guardar** essa informação? (2) ela tem **vários dados próprios**, ou é um valor só? (3) existem **vários exemplares** dela? (4) é **a mesma coisa** que outro candidato com outro nome?

Candidato	Decisão	Por quê
Programa	**Classe** Program	O sistema guarda centenas deles, cada um com nome, valor, tipo, região, site, status
Agência / quem oferece	**Classe** Provider	Uma agência oferece vários programas; como classe, o nome dela é guardado uma vez só
Critério	**Classe** EligibilityCriterion	Cada programa tem vários, e cada um tem comportamento próprio: saber se avaliar como "cabe" ou "não cabe"
Usuário / conta	**Classe** User	O sistema guarda e-mail, idioma, data de criação e o papel da conta (dono ou curador)
Perfil da empresa	**Classe** BusinessProfile, separada de User	É opcional (0..1), tem muitos dados próprios e o mesmo formato serve para as respostas anônimas do questionário
Consentimento	**Classe** Consent	Não é só um "sim": tem finalidade, data e pode ser retirado. A lei pede que isso seja registrado
Lista	**Classe** SavedList	Agrupa programas e tem comportamento (adicionar, remover, gerar relatório)
Resultado	**Classe** MatchResult	Junta um programa e um grupo (Match, Maybe, No)

Candidato	Decisão	Por quê
Motivo por critério	**Classe** CriterionResult	Cada resultado tem vários motivos, um por critério, cada um com sua marca e seu texto
Valor máximo, província, setor, e-mail, site oficial	**Atributos**	São valores simples, sem dados próprios
Relatório PDF	**Não** é entidade	É gerado na hora a partir da lista; não precisa ser guardado. Vira um objeto de fronteira (saída)
Questionário, pergunta	**Não** é entidade	É a tela por onde o usuário responde (objeto de fronteira); as respostas viram o perfil
Visitante	**Não** é classe	É um ator anônimo: o sistema não guarda nada sobre ele
Agência = provedor = organização	**Sinônimos**, uma classe só	Escolhi um nome (Provider) e registrei no dicionário de dados

Regra prática: se a coisa tem vários dados próprios, existem vários exemplares dela e o sistema precisa lembrar dela, é classe. Se é um valor só, é atributo. Se é tela, é fronteira. Se só coordena um caso de uso, é controle.

Passo 3: verbos viram operações

"Avaliar o perfil" virou `evaluate(profile)` em Program e EligibilityCriterion. "Salvar na lista" virou `add(p)`; "gerar relatório", `toReport()`; "excluir meus dados", `deleteAll()`; "retirar consentimento", `withdraw()`. A pergunta-chave é **de quem é a responsabilidade**: quem tem a informação necessária faz a operação. Quem sabe se um critério "cabe" é o próprio critério, porque é ele que conhece a regra.

Passo 4: ligar as classes e decidir o tipo de ligação

Para cada "tem" ou "pertence", fiz duas perguntas. **Quantos?** (vira a multiplicidade: um programa tem 1..* critérios). **A parte morre junto com o todo?** (sim = composição; não = agregação). Para cada "é um tipo de", usei herança: os critérios de província, porte e setor são tipos de critério que verificam de jeitos diferentes.

Passo 5: testar o modelo com os casos de uso

Por fim, "andei" por cada caso de uso perguntando: consigo fazer cada passo com essas classes? É esse teste que justifica a classe CriterionResult: com só um MatchResult guardando o grupo (Match, Maybe, No), o Find My Support não teria onde guardar **o motivo de cada critério**, que ele precisa mostrar na tela. O diagrama de sequência (seção 7) é exatamente esse teste, desenhado.

O mesmo teste encontrou outra lacuna, já depois do primeiro desenho. Ao percorrer o caso de uso "o curador entra no sistema", ficou claro que a classe User não tinha como dizer se uma conta é de dono de empresa ou de curador. Por isso User ganhou o atributo `role: Owner | Curator`. É assim que os modelos evoluem: cada caso de uso novo testa o que já foi desenhado.

Para a prova: o diagrama de classes de análise mostra os conceitos do problema, não o banco de dados nem o código. Os nomes são palavras do negócio. Além das classes de entidade (o que se guarda), existem as de fronteira (telas, PDF) e as de controle (quem coordena um caso de uso, como o MatchingController). O diagrama abaixo mostra só as de entidade.

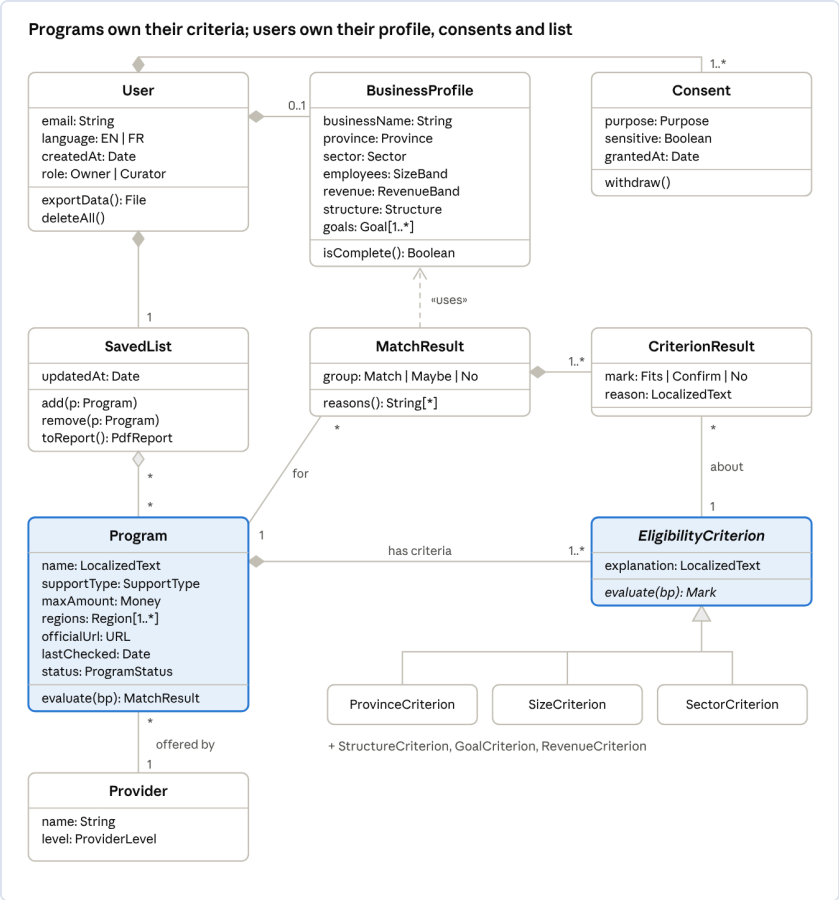


Figura 3. Diagrama de classes de análise: as classes de entidade do Maple Seed.

Por que estas escolhas

- **Composição (losango cheio) entre Programa e Critério.** Um critério como "só Ontario" não existe sozinho: se o programa for apagado, os critérios vão junto. O mesmo vale para o Usuário e seus dados pessoais, o que torna o "excluir meus dados" natural: apagar o usuário leva tudo junto.
- **Agregação (losango vazio) entre Lista salva e Programa.** O programa existe independentemente da lista e pode estar em várias listas.
- **Herança nos critérios (ProvinceCriterion, SizeCriterion...).** Esta é a decisão mais importante do diagrama. Cada tipo de critério verifica de um jeito diferente, mas o motor de matching trata todos igual: chama `evaluate(perfil)`. Isso é **polimorfismo**. Na prática, para adicionar um tipo de critério novo, cria-se uma subclasse sem mexer no motor.
- **Multiplicidade (0..1, 1..*, *).** Força decisões. Um usuário pode não ter perfil? Sim (0..1), porque pode ter criado a conta e ainda não preenchido.
- **Dicionário de dados.** Parece burocracia, mas evita que "Maybe" signifique uma coisa para quem desenhou e outra para quem programa. No Maple Seed ainda serve de base para o glossário em inglês e francês.

Para a prova: composição ou agregação? Pergunte: a parte morre junto com o todo? Se sim, composição (losango cheio). Se não, agregação (losango vazio).

7. Diagrama de sequência

Mostra quem manda mensagem para quem, e em que ordem, durante um caso de uso. Lê-se de cima para baixo, como uma linha do tempo.

Como cheguei a este diagrama

O diagrama de sequência não é inventado: ele é o **fluxo de eventos do caso de uso**, reescrito como mensagens entre os objetos do diagrama de classes. Ou seja, ele junta duas coisas que já existiam.

Passo 1: escolher o caso de uso

Não se desenha sequência para tudo. Escolhe-se o caso de uso mais importante ou mais complicado, onde vários objetos precisam colaborar. O UC3 Find My Support é o coração do produto e envolve perguntas, regras, programas e resultados. Já "ver detalhes do programa" é só buscar e mostrar, e o desenho não ensinaria nada.

Passo 2: escolher os participantes (as colunas)

O livro do Bruegge dá uma receita simples para isso, e a ordem das colunas segue a mesma lógica, da esquerda para a direita:

Regra	Participante no UC3	De onde veio
Primeira coluna: o ator que inicia o caso de uso	Visitor	Tabela do UC3: "ator que inicia: Visitante"
Um objeto de fronteira para cada tela que o ator usa	QuestionnaireForm e ResultsPage	As telas do protótipo: o questionário e a página de resultados
Um objeto de controle por caso de uso, para coordenar	MatchingController	Alguém precisa receber as respostas e organizar o trabalho
As entidades que têm a informação necessária	Program e EligibilityCriterion	O diagrama de classes: são eles que conhecem as regras

Passo 3: transformar cada passo do fluxo em mensagens

Peguei o fluxo de eventos do UC3, passo a passo, e perguntei: quem fala com quem para isso acontecer?

Passo do caso de uso	Mensagem no diagrama
1–2. O sistema mostra as perguntas e o visitante responde, cinco vezes	Visitor → Form: answer(question), dentro de um loop [5 questions]; Form → Visitor: next question
3. O sistema avalia a elegibilidade de cada programa ativo	Form → Controller: findMatches(answers); Controller → Program: evaluate(profile), num loop [each active program]
(detalhe do passo 3) cada critério é verificado	Program → Criterion: evaluate(profile), num loop [each criterion]; resposta: mark + reason
4. O sistema mostra os programas agrupados, com os motivos	Controller agrupa (mensagem para si mesmo); Controller → ResultsPage: show(results, checklist); ResultsPage → Visitor

Passo 4: decidir quem é responsável por cada coisa

Esta é a decisão mais importante. O controlador poderia, ele mesmo, abrir cada programa e verificar cada regra. Mas aí ele precisaria conhecer a lógica de todos os tipos de critério, e qualquer critério novo obrigaria a mexer nele. Por isso apliquei a regra do **especialista na informação**: quem tem a informação faz o trabalho. O programa conhece seus critérios, então ele os percorre. O critério conhece sua regra, então ele mesmo se avalia. O controlador só coordena.

Passo 5: conferir com o diagrama de classes

Toda mensagem que chega a um objeto precisa ser uma **operação da classe dele**. Se Program recebe evaluate(profile), a classe Program precisa ter essa operação. Fazer essa conferência revela o que falta: o diagrama de sequência mostrou que cada critério devolve "marca + motivo", e por isso a classe CriterionResult precisa existir.

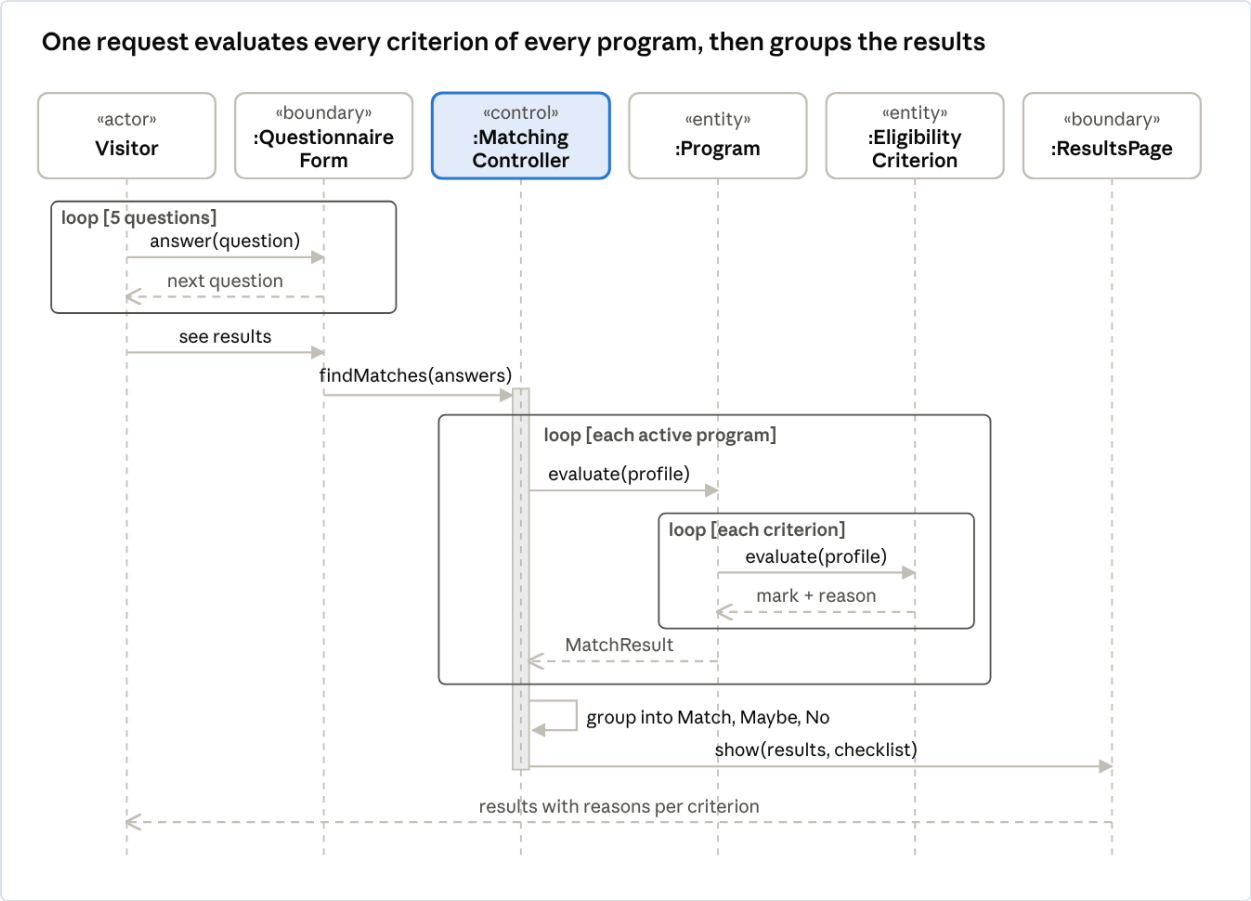


Figura 4. Diagrama de sequência do caso de uso UC3, Find My Support.

- **Por que este caso de uso:** o Find My Support é o coração do produto e o que tem a lógica mais interessante. Diagramar "ver detalhes do programa" seria trivial.
- **O que ele ensinou:** o controlador **não** verifica regra nenhuma; ele pede ao Programa, que pede a cada Critério. Isso mantém a lógica num lugar só e fácil de testar. Também mostrou que a mesma operação `evaluate` serve para a busca e para as recomendações.

Para a prova: setas cheias são pedidos, setas tracejadas são respostas, e as caixas "loop" repetem o que está dentro delas. Muitas vezes o diagrama de sequência revela operações que faltavam no diagrama de classes.

8. Máquinas de estado

Mostram os estados possíveis de um objeto e os eventos que o fazem mudar de estado. Só vale a pena desenhar uma quando o objeto tem um ciclo de vida que importa.

Como cheguei a estes diagramas

Enquanto o diagrama de sequência acompanha **vários objetos durante um caso de uso**, a máquina de estado acompanha **um objeto durante toda a vida dele**, atravessando vários casos de uso.

Passo 1: escolher quais objetos merecem uma máquina de estado

Procurei objetos cujo comportamento depende da situação em que estão. Pistas no texto: palavras como "status", "ainda não", "depois de", "volta para", "enquanto". Os candidatos foram quatro:

Candidato	Decisão	Por quê
Sessão do questionário	Desenhar	Avança pergunta a pergunta, pode voltar, editar, recomeçar. Fácil de programar errado
Programa	Desenhar	Muda de situação quando os dados do governo mudam, e isso decide se ele pode ser recomendado
Conta do usuário	Deixar para depois	Tem estados (visitante, registrado, excluído), mas são poucos e simples; ficou anotada para o design detalhado
Lista salva, critério	Não desenhar	Não mudam de comportamento: uma lista sempre aceita adicionar e remover

Passo 2: encontrar os estados

Um estado é uma situação em que o objeto **reage de forma diferente** aos mesmos eventos. A pergunta que fiz foi: "em que situações esse objeto responde diferente?"

- **Sessão:** antes de começar, o botão "começar" funciona; respondendo, o que funciona é "responder" e "voltar"; nos resultados, "editar uma resposta" e "recomeçar". São três estados. Em vez de criar cinco estados (pergunta 1, pergunta 2...), usei um só, "Respondendo a pergunta n", com um contador n. Isso deixa o desenho menor sem perder nada.
- **Programa:** os três estados vieram de requisitos que já existiam. "Ativo" e "Fechado" vêm dos dados do governo (aceita ou não candidaturas). "Precisa revisão" vem do UC11 ("programas alterados são marcados para revisão") e do princípio de ser honesto: um programa com dados novos não pode ser recomendado com regras antigas.

Passo 3: encontrar os eventos e as transições

Os eventos que fazem o objeto mudar de estado vêm dos passos e dos fluxos alternativos dos casos de uso. "O visitante volta para mudar uma resposta" (fluxo alternativo do UC3) virou a transição "edit an answer". "O curador confirma as regras" (UC12) virou "rules checked". Quando a mesma ação leva a lugares diferentes dependendo de uma condição, usa-se a **guarda** entre colchetes: responder leva à próxima pergunta [n < 5] ou aos resultados [n = 5]. E o que o sistema faz na passagem vira a **ação** depois da barra: "/ evaluate all programs".

Passo 4: conferir se falta alguma coisa

Para cada estado, pergunte: "e se acontecer cada um dos outros eventos aqui?" Essa conferência mostra os buracos. Por exemplo: e se um programa for encerrado enquanto está em "Precisa revisão"? O diagrama ainda não responde. Isso não é um erro do método; é justamente o método mostrando uma pergunta que precisa ser decidida antes de programar.

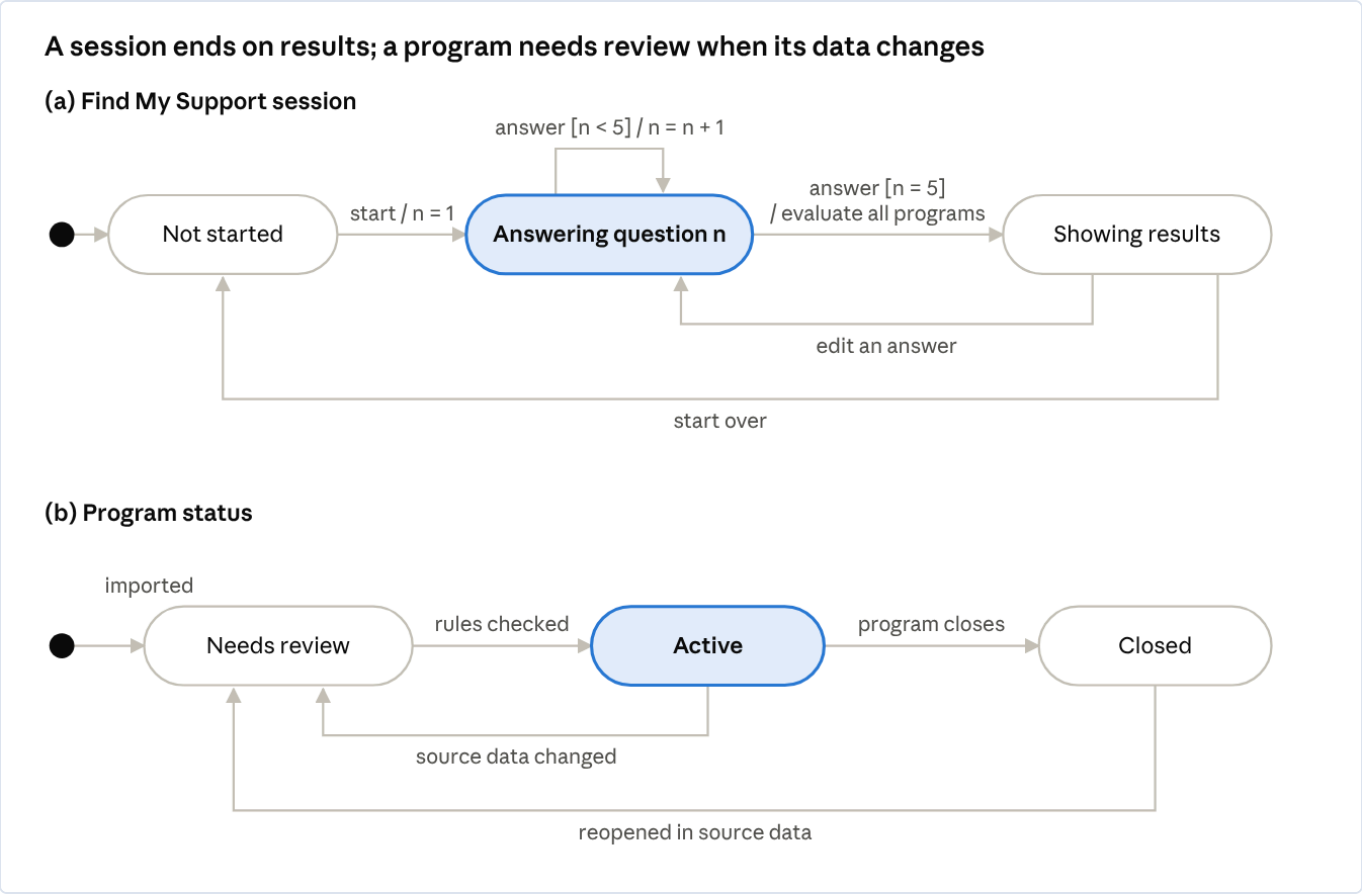


Figura 5. Máquinas de estado da sessão do questionário (a) e do status do programa (b).

- **A sessão do questionário:** perguntas 1 a 5, depois resultados, com volta para editar uma resposta ou recomeçar. Sem esse desenho é fácil esquecer de programar o "editar uma resposta e recalcular".
- **O status do programa:** "Precisa revisão", "Ativo" e "Fechado". Essa máquina traduz um princípio de negócio, o de ser honesto, em regra de software: um programa cujos dados mudaram nunca é recomendado com regras antigas sem passar por revisão.

Para a prova: os rótulos das transições seguem o formato *evento [condição] / ação*. Exemplo: "answer [n = 5] / evaluate all programs".

9. Arquitetura

Mostra as grandes partes do sistema, como elas conversam e onde rodam. Dois conceitos guiam tudo: coesão alta e acoplamento baixo.

- **Coesão alta:** cada parte faz uma coisa só. O módulo de Matching só faz matching; não sabe nada de PDF nem de login.
- **Acoplamento baixo:** as partes dependem pouco umas das outras e só conversam por "portas" definidas (as interfaces). Assim dá para trocar o banco de dados sem quebrar o relatório.

Como cheguei a esta arquitetura

A arquitetura junta tudo o que veio antes: os requisitos dizem **o que a arquitetura precisa garantir**, e as classes dizem **o que vai dentro de cada parte**.

Passo 1: começar pelos requisitos que mais pesam

Os requisitos não funcionais são os que mais decidem a arquitetura. Para cada um, perguntei: que decisão de estrutura ele obriga?

Requisito	Decisão de arquitetura que ele levou
Acessível pelo navegador, para muita gente	Aplicação web: cliente-servidor em camadas
Privacidade e "excluir meus dados"	Dados pessoais numa base separada dos dados públicos
Resultados explicáveis e testáveis	O matching isolado num subsistema próprio, sem depender de telas
Dados do governo atualizados com frequência	Uma linha de importação separada, usada só pelo curador
Bilíngue	Textos guardados nos dois idiomas no repositório e traduções nas telas

Passo 2: agrupar as classes em subsistemas

Um subsistema é um grupo de classes que trabalham juntas. Para formar os grupos, olhei **quais classes são usadas juntas nos mesmos casos de uso**. Classes que andam juntas ficam no mesmo subsistema (isso é coesão alta).

Subsistema	Classes	Casos de uso
Catalogue	Program, Provider	UC1 Buscar, UC2 Ver detalhes
Matching	EligibilityCriterion e subclasses, MatchResult, CriterionResult	UC3 Find My Support, UC13 Recomendações
Accounts	User, BusinessProfile, Consent, SavedList	UC4 a UC7, UC9, UC10
Reporting	PdfReport	UC8 Gerar PDF
Data import	ImportController e as etapas de importação	UC11 Importar, UC12 Manter regras

Passo 3: decidir quem conversa com quem

Depois, defini as "portas" de cada subsistema (as interfaces) e a direção das dependências. O Matching precisa dos programas do Catalogue, mas o Catalogue não precisa saber nada do Matching. O Reporting pede dados ao Catalogue e ao Accounts, mas ninguém depende do Reporting. A regra é evitar dependências em círculo (A depende de B, que depende de A), porque elas fazem uma mudança se espalhar pelo sistema inteiro.

Passo 4: escolher o estilo pela "forma" do problema

Cada estilo arquitetural resolve um tipo de problema. A escolha é comparar a forma do problema com a de cada estilo:

- Muitos usuários num navegador, um servidor central → **cliente-servidor** e **três camadas**.
- Muitas telas que mostram e alteram os mesmos dados → **MVC**.
- Vários subsistemas lendo os mesmos programas → **repositório**.
- Uma planilha que passa por várias transformações em sequência → **pipe-and-filter**.
- Nenhum dono central dos dados → **peer-to-peer**. Não é o nosso caso, por isso foi descartado.

Passo 5: colocar cada parte num lugar físico

Por último, decidi onde cada parte roda (os "nós"): o navegador do usuário, o servidor da aplicação e o servidor de banco de dados. A preferência por hospedagem no Canadá vem do requisito de privacidade.

Passo 6: testar a arquitetura com os casos de uso

Assim como nos outros diagramas, percorri os casos de uso pela arquitetura. UC3: navegador → controlador → Matching → repositório de programas. UC10 (excluir meus dados): navegador → Accounts → base de dados pessoais, e só ela. Se um caso de uso precisasse atravessar partes de um jeito estranho, seria sinal de que a divisão estava errada.

Dica: toda decisão de arquitetura é uma troca (ganha-se uma coisa, perde-se outra). Por isso vale registrar o motivo de cada uma, como foi feito aqui. No mercado, esse registro se chama ADR (Architecture Decision Record).

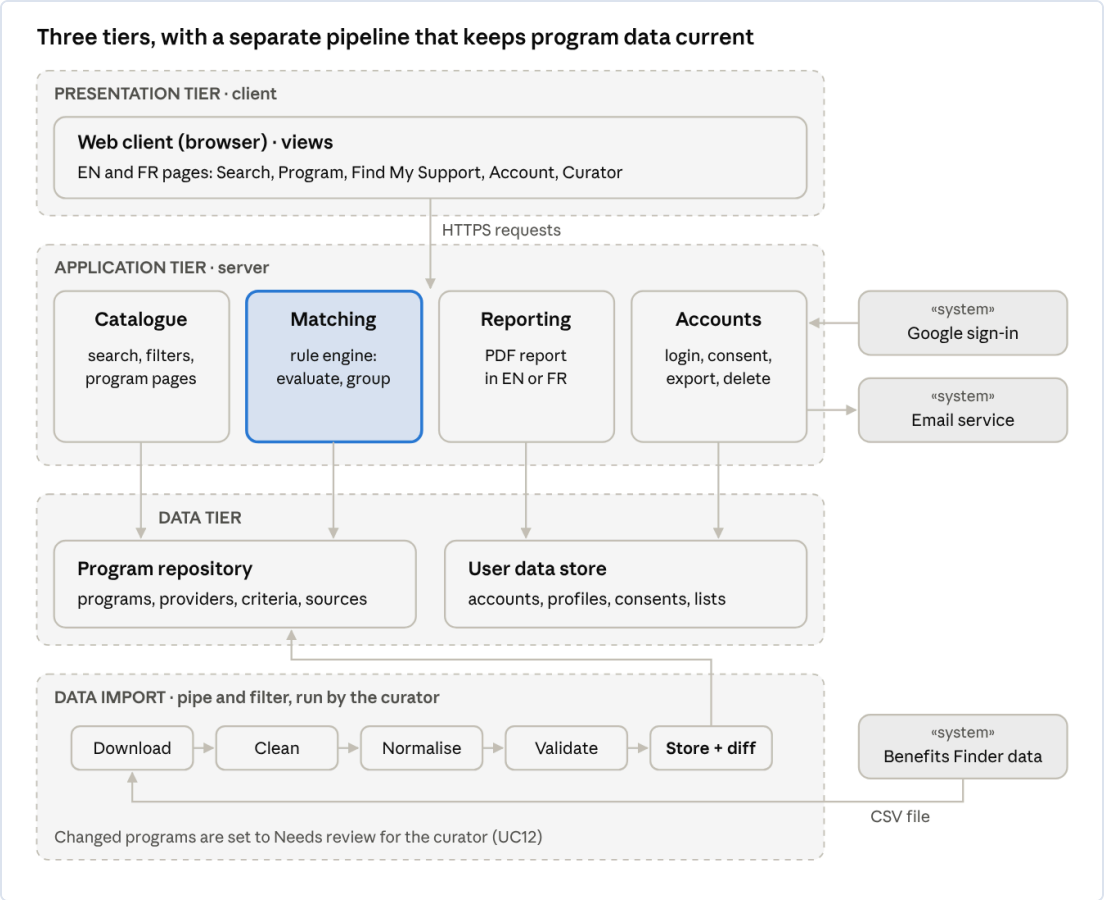


Figura 6. Arquitetura proposta: camadas, subsistemas, repositórios e sistemas externos.

Por que estes estilos arquiteturais

Estilo	O que é	Por que (ou por que não)
Três camadas (N-tier) e cliente-servidor	Tela, lógica e dados separados; vários clientes falam com um servidor	É o padrão para aplicações web
MVC	Views mostram, Models guardam, Controllers coordenam	Continuação direta da análise: as telas são Views, as classes do diagrama são Models, e os objetos de controle (como o MatchingController) são Controllers
Repositório	Várias partes leem uma base central	Busca, matching e relatório leem a mesma base de programas
Pipe-and-filter	Os dados passam por uma linha de etapas independentes	Importação: baixar, limpar, normalizar, validar, gravar. Cada etapa é testável sozinha, e a planilha do governo vai precisar de muita limpeza
Peer-to-peer	Todos os nós são cliente e servidor ao mesmo tempo	Descartado: o projeto precisa de uma fonte central e confiável de informação

Uma decisão que vale destacar: **os dados pessoais ficam numa base separada dos dados públicos**. Isso simplifica a privacidade e torna o "excluir meus dados" seguro e fácil.

10. O fio que liga tudo: rastreabilidade

Requisito → caso de uso → classes envolvidas → código → teste.

O melhor dessa abordagem é que tudo se conecta. As condições de entrada e saída de cada caso de uso viram casos de teste quase prontos. Se alguém perguntar "por que esse código existe?", você aponta o requisito. E se um requisito mudar, você sabe exatamente o que precisa mexer.

Resumo para revisão

Termo	Em uma frase	Exemplo no Maple Seed
Elicitação	Tirar do cliente o que ele realmente precisa	Hipóteses de dor e protótipo de telas
Requisito funcional	O que o sistema faz	Gerar o PDF da lista
Requisito não funcional	Como o sistema precisa ser	Bilíngue, acessível, privado
Ator	Um papel fora do sistema que interage com ele	Visitante, Curador, Google sign-in
Cenário	Uma história concreta, com nome	Maria filtra por Ontario
Caso de uso	A generalização de vários cenários	UC1 Buscar programas
Fronteira do sistema	O que está dentro e o que está fora	A candidatura fica fora
«include»	Comportamento sempre reutilizado	Find My Support inclui Avaliar elegibilidade
«extend»	Comportamento opcional	Login com Google estende Log in
Composição	A parte morre com o todo (losango cheio)	Programa e seus critérios
Agregação	A parte vive sozinha (losango vazio)	Lista salva e programas
Herança	"É um tipo de" (triângulo vazio)	ProvinceCriterion é um EligibilityCriterion
Polimorfismo	Tipos diferentes respondem à mesma operação	Todo critério responde a evaluate(perfil)
Multiplicidade	Quantos objetos participam da relação	Um usuário tem 0..1 perfil
Diagrama de sequência	Mensagens entre objetos ao longo do tempo	O fluxo do Find My Support
Máquina de estado	Estados de um objeto e o que os muda	Status do programa
Coesão	Cada parte faz uma coisa só (quer-se alta)	Matching não sabe de PDF
Acoplamento	Quanto as partes dependem umas das outras (quer-se baixo)	Partes falam só por interfaces
Rastreabilidade	Ligar requisito a código e teste	UC10 vira o teste de "excluir meus dados"